

أساسيات نظام ROS

اسم المقرر: أنظمة تشغيل الروبوتات | **الأسبوع الثالث** | مدة الشريحة: 2 ساعة

د. سلطان النفيعي - 2026

مراجعة سريعة — ماذا تعلمنا؟

01

ROS = Robot Operating System

نظام التشغيل المخصص للروبوتات

02

تطبيقات ROS

روبوتات في الصناعة والطب والنقل

03

توزيعات ROS

Noetic و Humble

04

ROS 2 مقابل ROS 1

الفروقات والتطور

05

مصادر التعلم

المواقع والوثائق الرسمية

ما سنتعلمه اليوم

١. مستويات النظام

هيكل ROS الثلاثي المستويات

٢. نظام الملفات

Filesystem Levelوالحزم

٣. شبكة المعالجة

Computation Graph Level

٤. الاتصال

Messages، Services، Topics

٥. المكتبات والأدوات

Gazebo، roscpp، rospy

هيكل ROS — مستويات النظام

Application Level

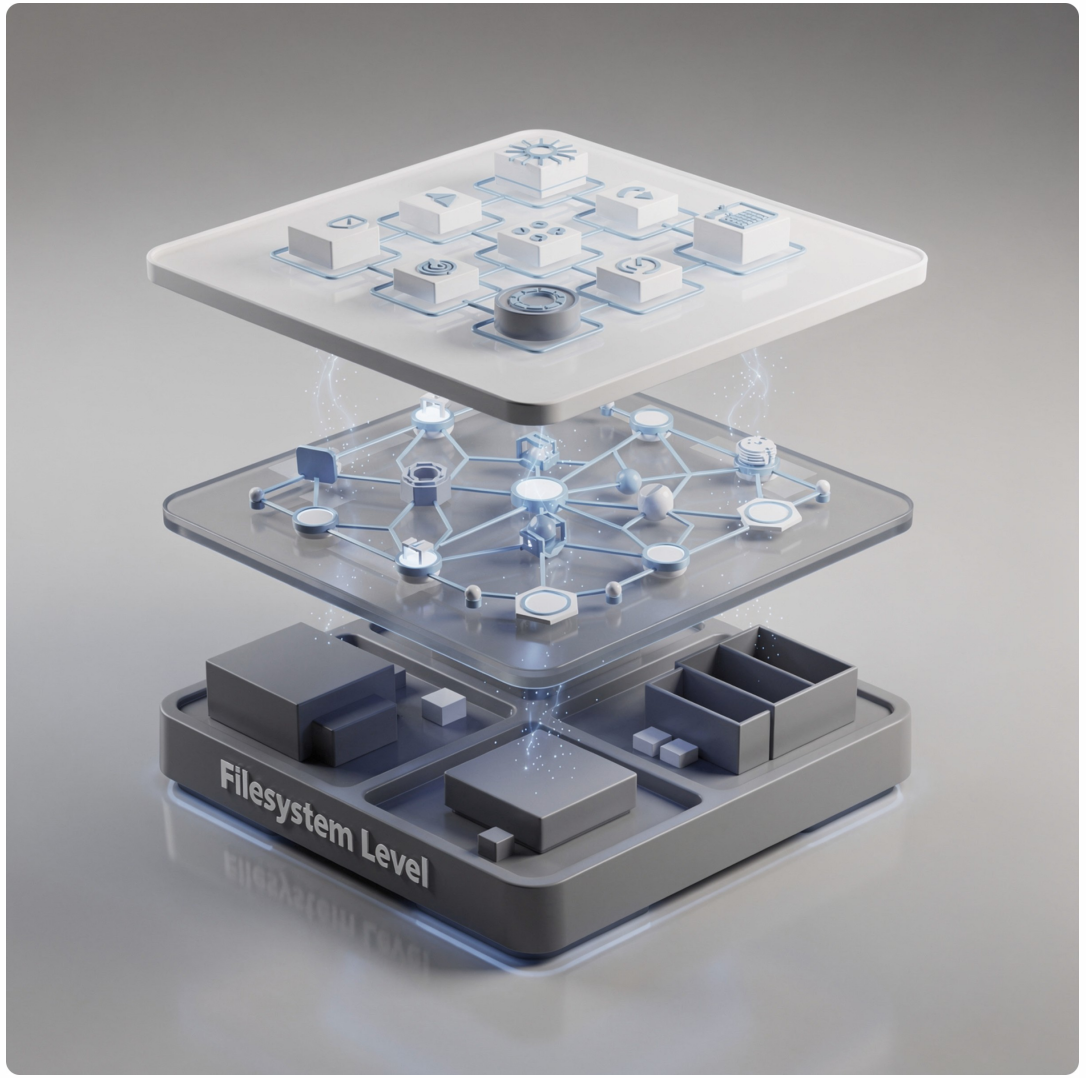
التطبيقات والأدوات الجاهزة للاستخدام

Computation Graph Level

شبكة العقد والتواصل بينها

Filesystem Level

تنظيم الملفات والحزم البرمجية



مستوى نظام الملفات — Filesystem Level

تنظيم الملفات

نظام واضح لتنظيم كل ملفات ROS

هيكل المجلدات

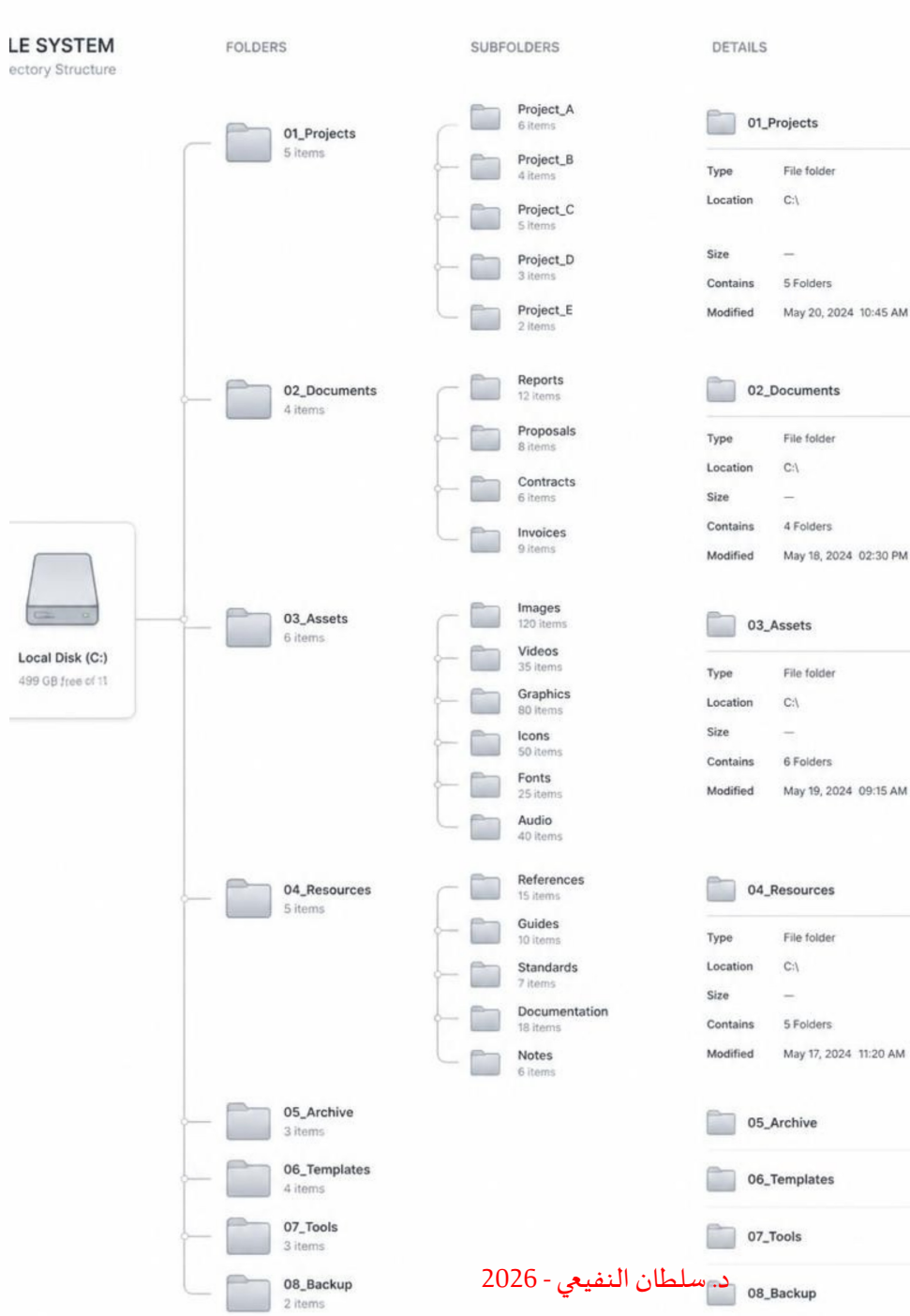
بنية موحدة لجميع مشاريع ROS

أنواع الملفات

XML ،CMake ،Python ،C++

توزيع الحزم

كيفية تنظيم الحزم البرمجية وتوزيعها



د. سلطان النفيعي - 2026

هيكل المجلدات — Directory Structure

`/catkin_ws/src/`

كود المصدر — تُوضع هنا حزمك البرمجية

`/catkin_ws/build/`

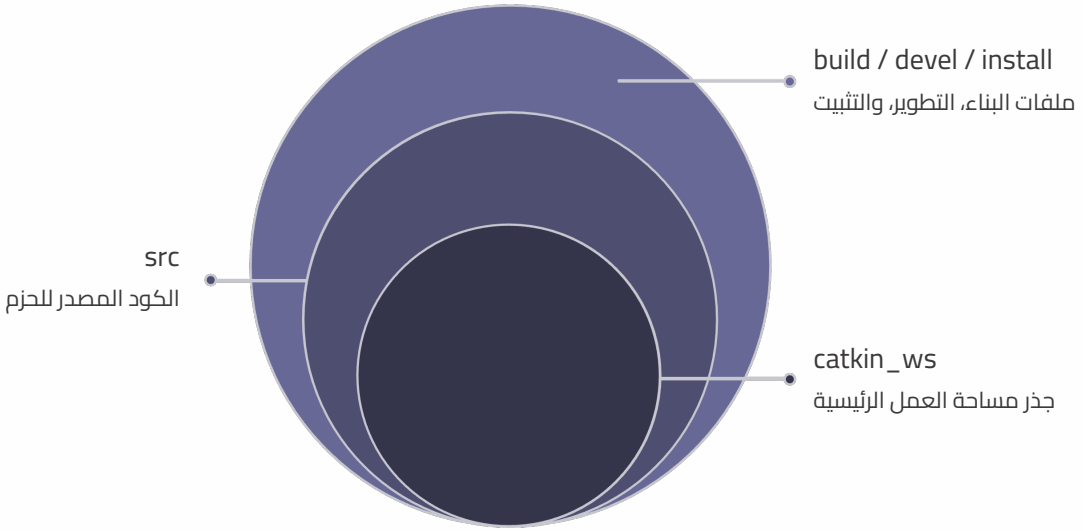
ملفات البناء — تُنشأ تلقائياً عند البناء

`/catkin_ws/devel/`

ملفات التطوير — البيئة المؤقتة للتشغيل

`/catkin_ws/install/`

ملفات التثبيت — النسخة النهائية





ما هو Catkin Workspace؟

مجلد خاص لمشاريع ROS

بيئة عمل مخصصة تحتوي على جميع مشاريعك في مكان واحد

يحتوي على حزمك البرمجية

كل مشروع أو وظيفة تصبح حزمة مستقلة داخل مجلد src

ينظم الكود ويسهل البناء

نظام catkin_make يبني جميع الحزم بأمر واحد

مجلد src — المصدر Source

مجلد `src` هو قلب مساحة العمل — هنا يعيش كود روبوتاتك.

حزمة منفصلة لكل وظيفة

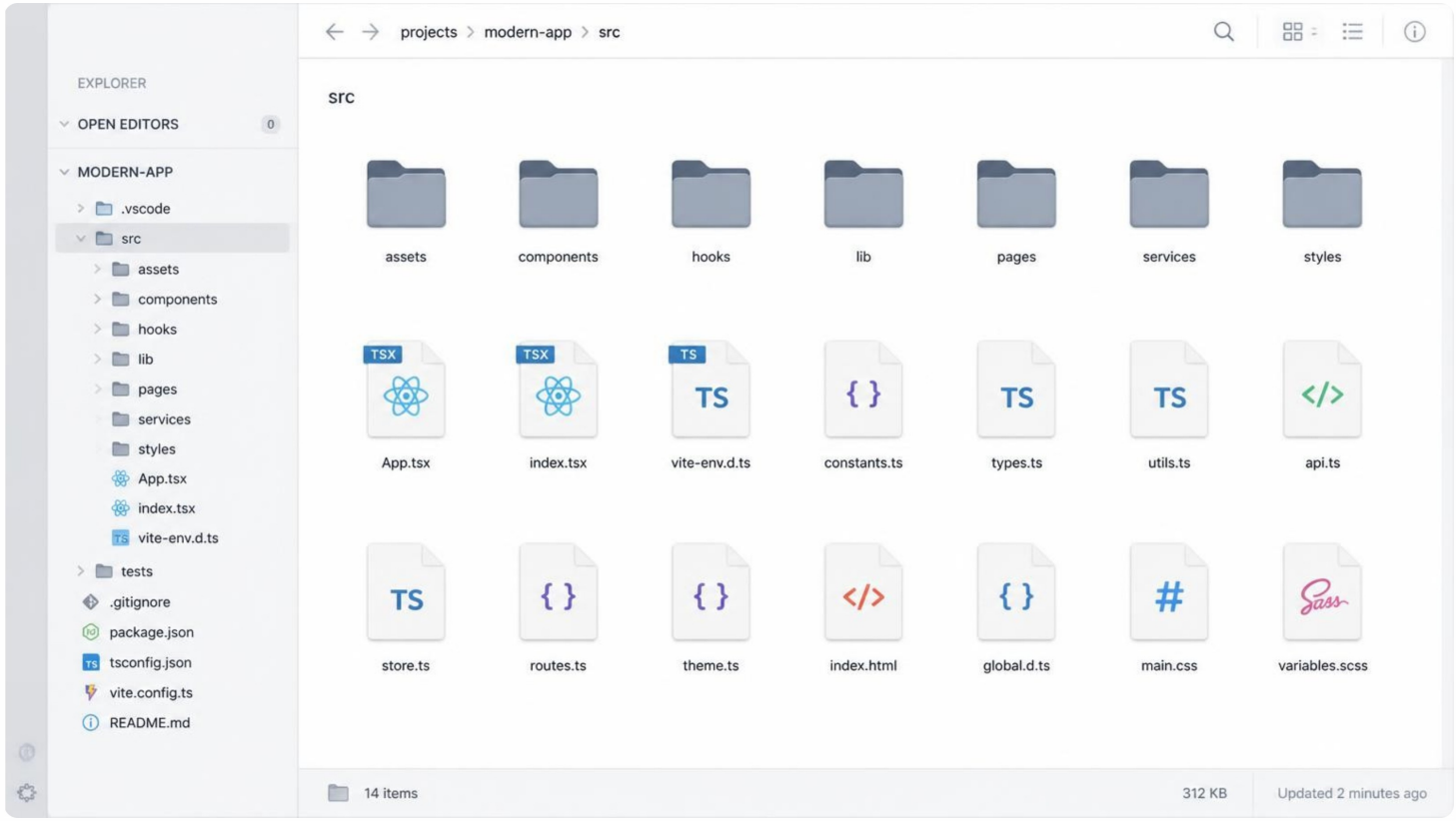
كل حزمة في مجلد مستقل داخل src

كود البرمجة

ملفات Python أو C++ لتنفيذ المنطق البرمجي

ملفات الإعداد والتبعيات

مطلوبان CMakeLists.txt و package.xml في كل حزمة



هيكل الحزمة — Package Structure

`launch/`

ملفات التشغيل التلقائي

`include/`

ملفات الرأس Header Files

`src/`

كود Python أو C++

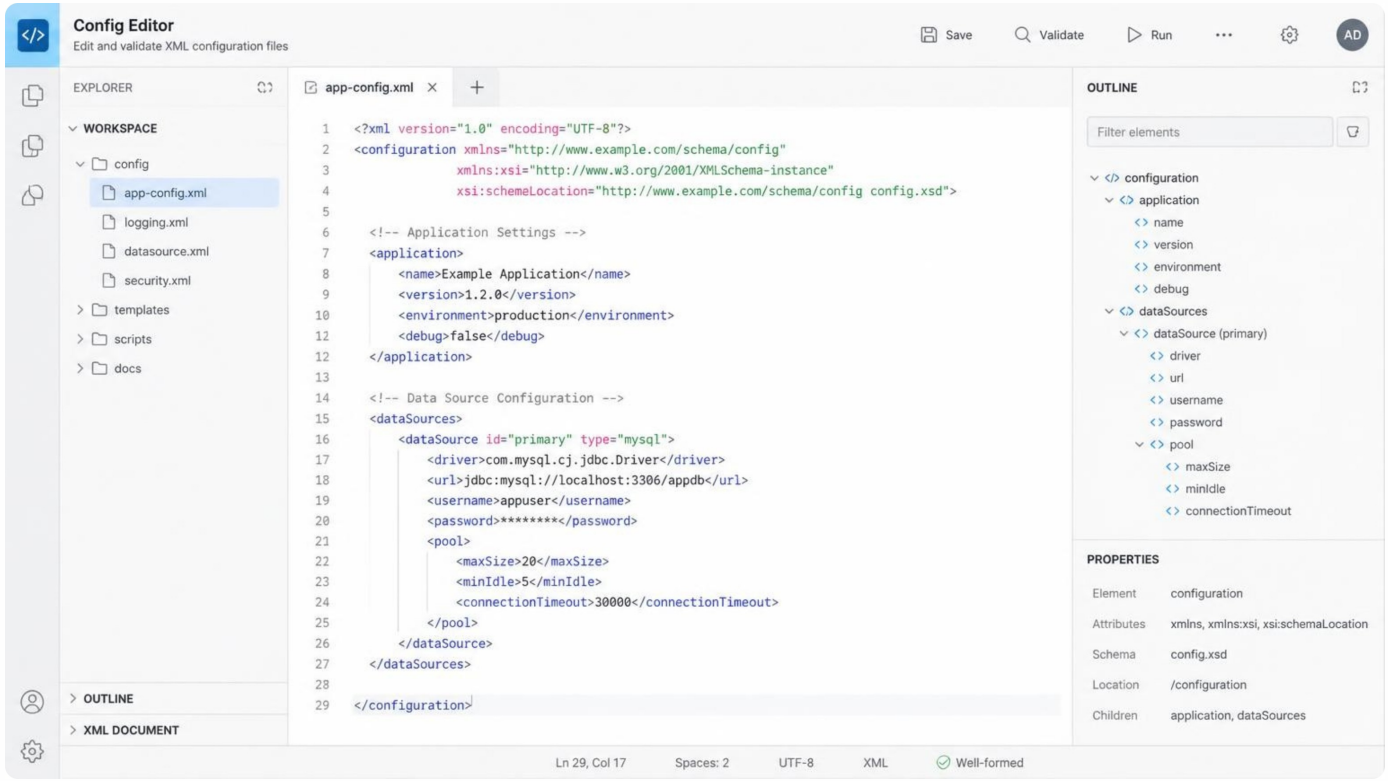
`package.xml`

وصف الحزمة والتبعيات

`cfg/`

ملفات الإعداد والضبط

ملف package.xml — وصف الحزمة



ملف `package.xml` هو هوية الحزمة — يصف من أنت وماذا تحتاج.

الاسم والإصدار

تعريف الحزمة بشكل فريد في النظام

المؤلف والترخيص

معلومات المطور وحقوق الاستخدام

التبعيات

الحزم الأخرى التي يحتاجها هذا المشروع

ملف CMakeLists.txt — ملف البناء

1

تعليمات البناء

يخبر النظام كيف يبني الحزمة خطوة بخطوة

2

المكتبات المطلوبة

تحديد الحزم الخارجية اللازمة للبناء

3

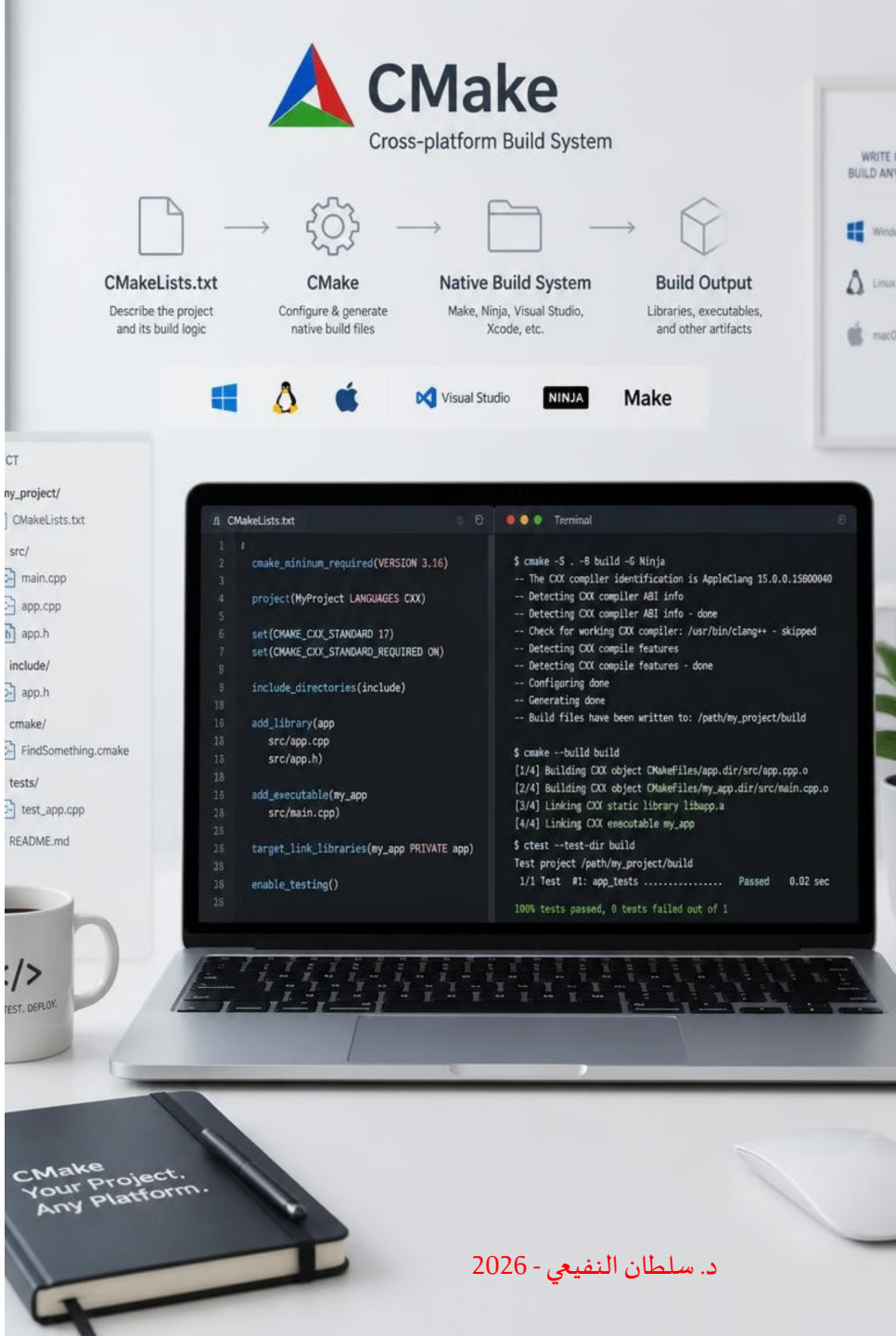
الملفات المراد تجميعها

قائمة ملفات C++ أو Python المستهدفة

4

الأهداف المراد إنشاؤها

الملفات التنفيذية والمكتبات الناتجة



د. سلطان النفيعي - 2026

مستوى شبكة المعالجة — Computation Graph Level

شبكة المعالجة هي العمود الفقري لنظام ROS — تصف كيف تتحدث أجزاء النظام مع بعضها.

تفاعل العقد

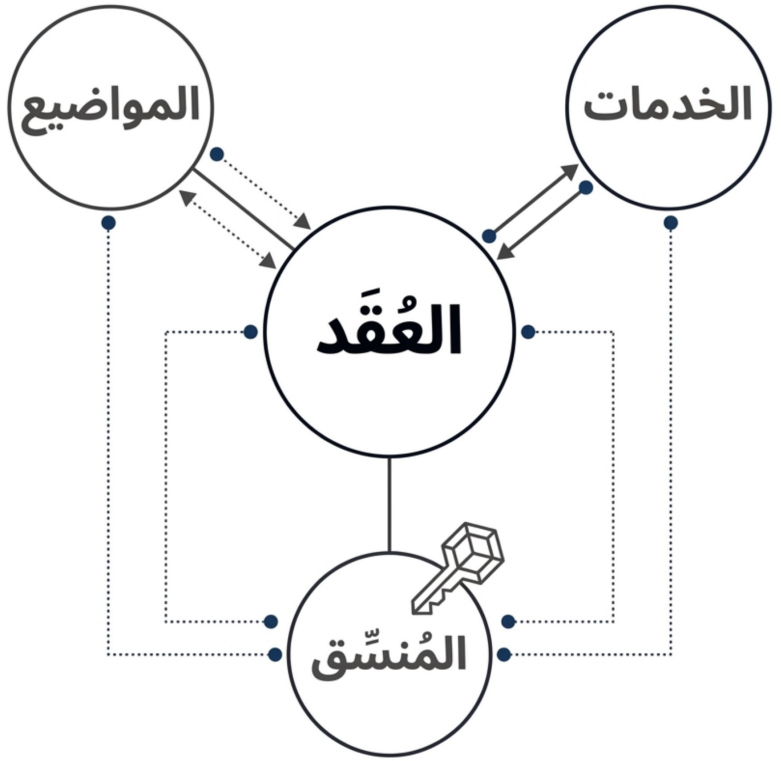
كل عقدة تؤدي وظيفة محددة وتتواصل مع غيرها

تدفق البيانات

البيانات تنتقل بشكل منظم عبر المواضيع

الرسم البياني للنظام

يمكن رؤية كل الاتصالات بصرياً بأداة rqt_graph



مكونات الشبكة — Graph Components

1 العقد — Nodes

وحدات البرمجة المستقلة

2 الرسائل — Messages

البيانات المتبادلة بين العقد

3 المواضيع — Topics

القنوات الناقلة للبيانات

4 الخدمات — Services

الطلب والاستجابة الثنائية

5 الرئيسي — Master

مركز إدارة الشبكة كلها

د. سلطان النفيعي - 2026



Sensor Node

قراءة بيانات الحساسات



Motor Node

التحكم بالمحرك والحركة



Navigation Node

حساب المسار والتوجيه



Camera Node

قراءة الكاميرا ونقل الصور

العقدة — Node

برنامج صغير مستقل يؤدي وظيفة واحدة محددة داخل منظومة ROS

الرسائل — Messages

الرسائل هي الطريقة الوحيدة لنقل البيانات بين العقد في ROS — لها أنواع معيارية محددة.

`geometry_msgs/Twist`

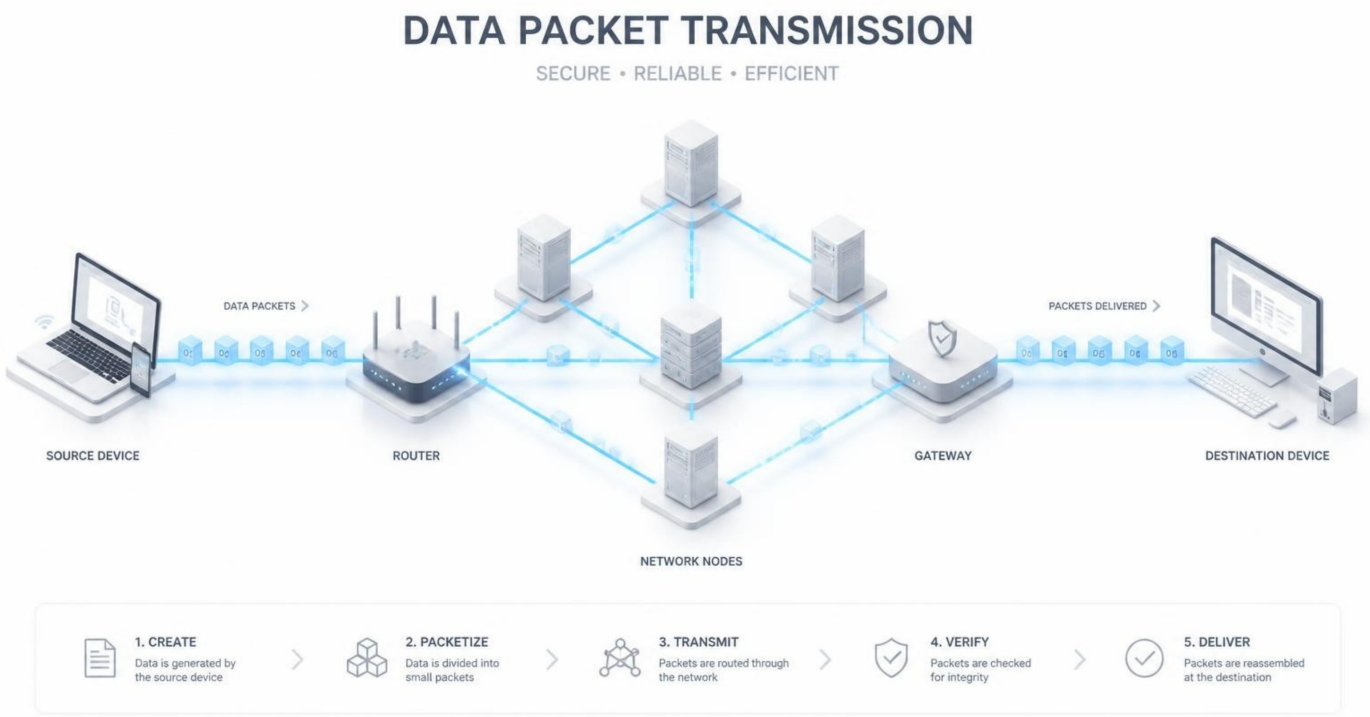
السرعة الخطية والزاوية للروبوت

`sensor_msgs/LaserScan`

بيانات مسح الليزر والمسافات

`std_msgs/String`

رسائل نصية بسيطة



المواضيع — Topics

1

Publisher Node

العقدة التي تنشر البيانات على الموضوع

2

/topic_name

القناة الناقلة — لها اسم ونوع رسالة محدد

3

Subscriber Node

العقدة التي تستقبل وتستهلك البيانات

النشر والاشتراك غير متزامنين — العقد لا تنتظر بعضها



مثال على Topic — موضوع السرعة

Node A — مُحدد السرعة

يحسب السرعة المطلوبة وينشرها على الموضوع `cmd_vel/`

cmd_vel/ — الموضوع الناقل

القناة الموحدة لنقل أوامر السرعة في ROS

Node B — منفذ السرعة

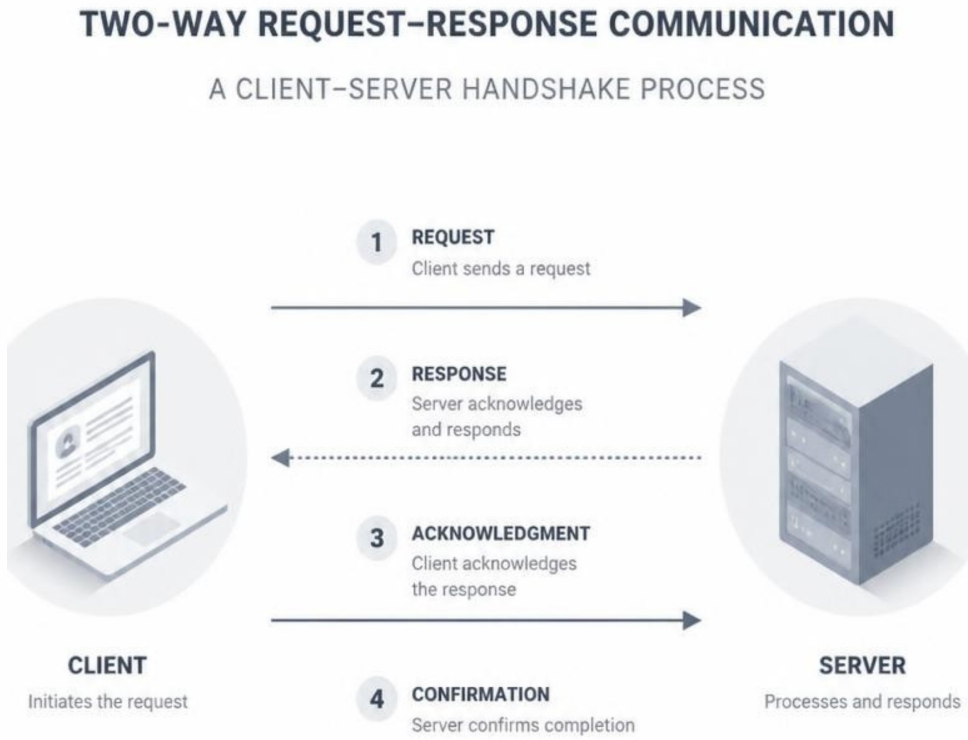
يستقبل قيم السرعة ويُرسلها للمحركات فعلياً

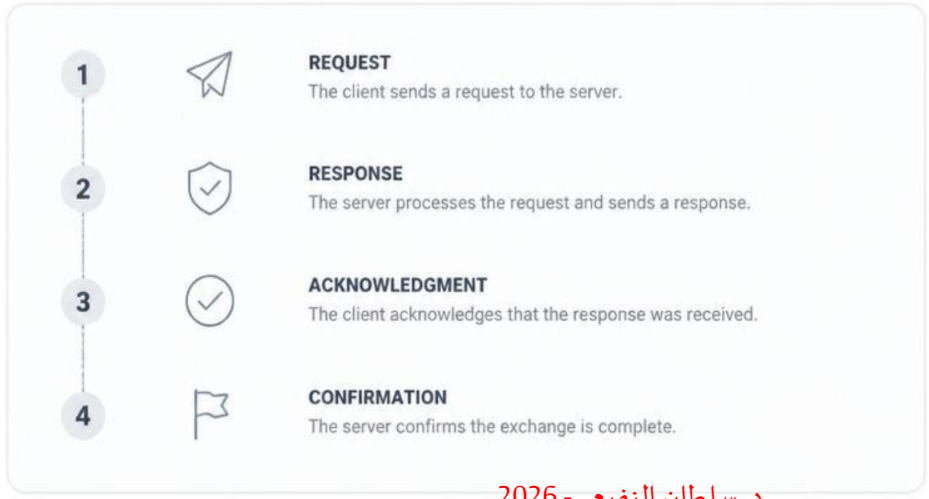


الخدمات — Services

الطلب — Request	الانتظار
العميل يرسل طلباً محدداً للخادم	العميل يتوقف حتى يصل الرد — اتصال متزامن
الاستجابة — Response	
الخادم يُنجز المهمة ويُعيد نتيجة	

مثال عملي: "تحرك مسافة متر واحد إلى الأمام" ← ينفذ الروبوت الأمر ← يرد: "تم التحرك بنجاح"





الفرق بين Topic و Service

Service — الخدمة	Topic — الموضوع
(طلب و استجابة) اتجاهان	(نشر واشتراك) اتجاه واحد
مرة واحدة ومتزامن	مستمر وغير متزامن
ينتظر حتى يصل الرد	لا انتظار للرد
تحرك لمسافة محددة :مثال	سرعة الروبوت المستمرة :مثال

الرئيسي — ROS Master

ROS Master هو القلب النابض للنظام — بدونه لا تعمل أي عقدة.

يدير الشبكة بالكامل

نقطة التحكم المركزية لجميع الاتصالات

يسجل العقد ويربطها

يربط الناشرين بالمشتركين تلقائياً

شرط أساسي

بدون `roscore` لا يعمل أي شيء في ROS



Parameter Server — خادم المعاملات

/robot_name

اسم الروبوت

"MyRobot" — قيمة نصية يصل إليها الجميع

/max_speed

السرعة القصوى

2.5 م/ث — قيمة رقمية للتحكم في الحركة

/battery

مستوى البطارية

85% — قيمة يقرأها جميع المكونات

خادم المعاملات يُخزّن إعدادات النظام كقيم ثابتة يمكن لجميع العقد قراءتها وتعديلها في أي وقت.

د. سلطان النفيعي - 2026

الاتصال بين الأجزاء — Communication

شبكة العقد

العقد تتواصل عبر بروتوكولات الشبكة القياسية

بروتوكول TCP/IP

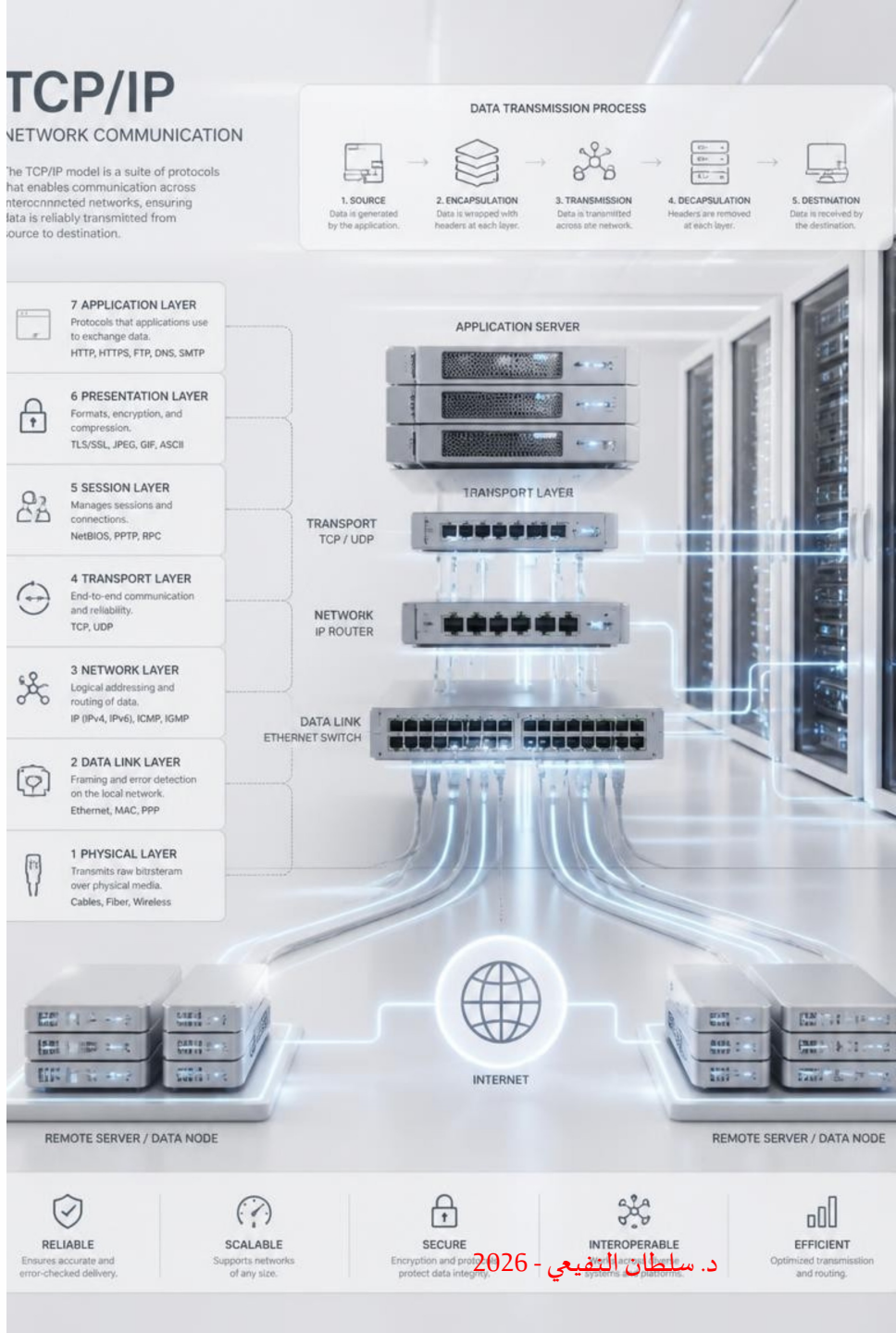
للاتصالات الموثوقة التي تتطلب ضماناً للوصول

بروتوكول UDP

للبيانات سريعة الزوال كتدفق الصور

شفافية كاملة

ROS يتولى كل تفاصيل الشبكة — المبرمج لا يحتاج لمعرفتها



د. سلطان التميمي - 2026

رسم بياني للنظام — Graph Visualization

أداة: `rqt_graph`

رؤية بصرية للشبكة

ترسم جميع العقد والمواضيع كرسم بياني تفاعلي

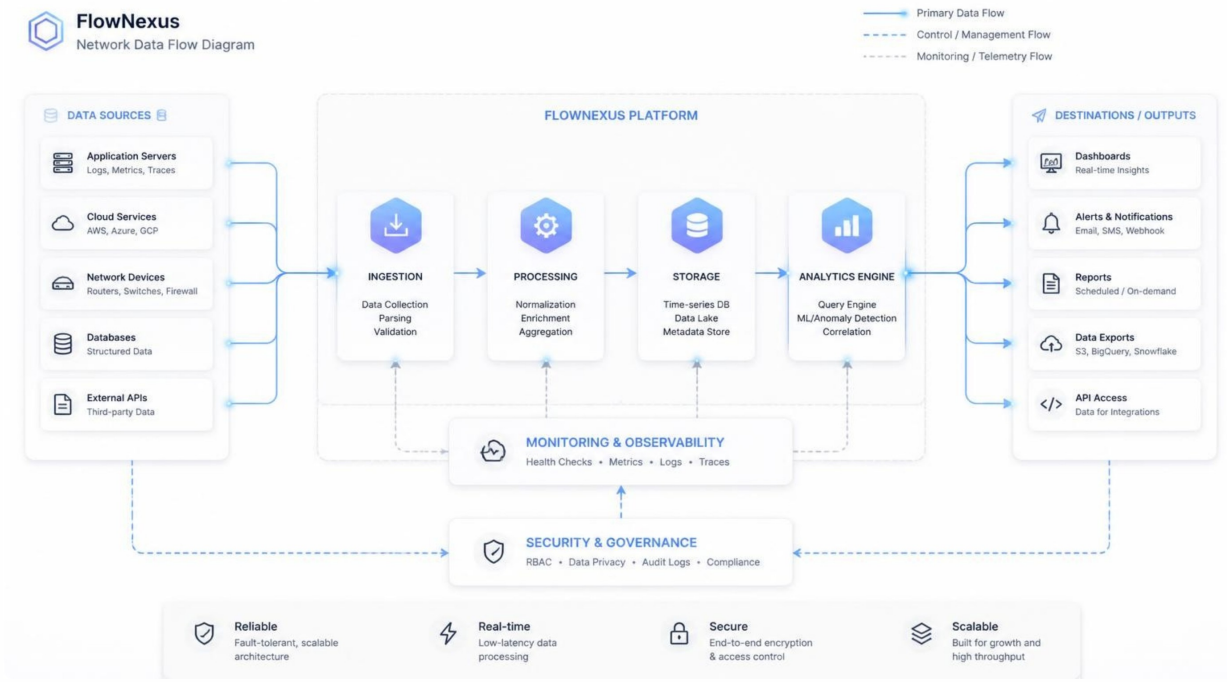
تدفق البيانات

تُظهر من يُرسل ومن يستقبل البيانات في الوقت الفعلي

أداة التشخيص

تساعد في إيجاد الأخطاء وحل مشاكل الاتصال بسرعة

الأمر: `rosrun rqt_graph rqt_graph`



المكتبات — Libraries

roscpp

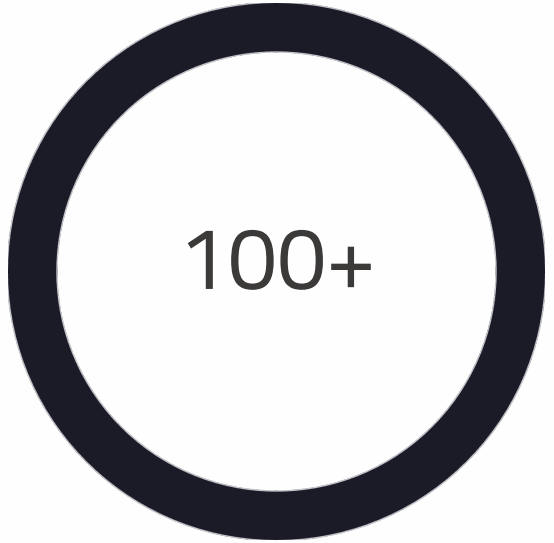
مكتبة C++ لـ ROS

أعلى أداءً ومناسبة للتطبيقات الحقيقية عالية السرعة

rospy

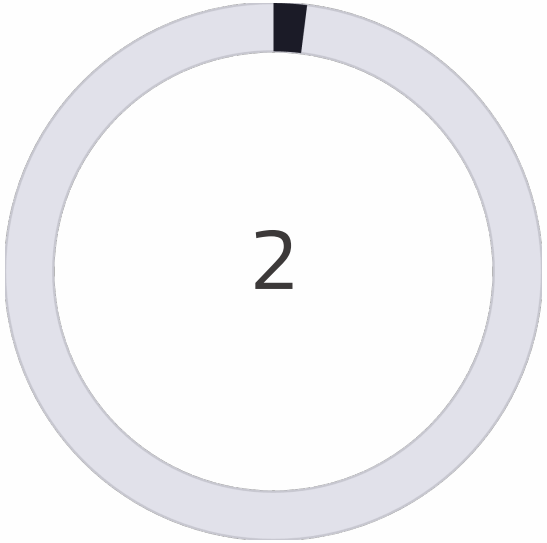
مكتبة Python لـ ROS

سهلة التعلم ومثالية للنماذج الأولية والتعليم



حزمة جاهزة

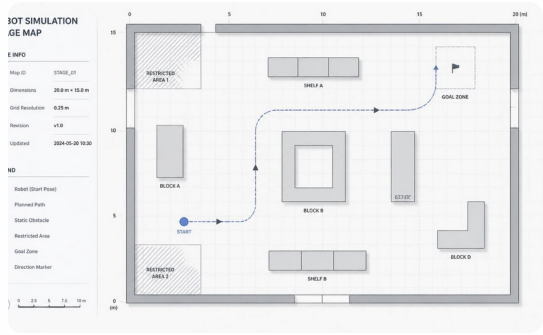
في مستودع ROS تغطي معظم احتياجات الروبوتات



لغات رئيسية

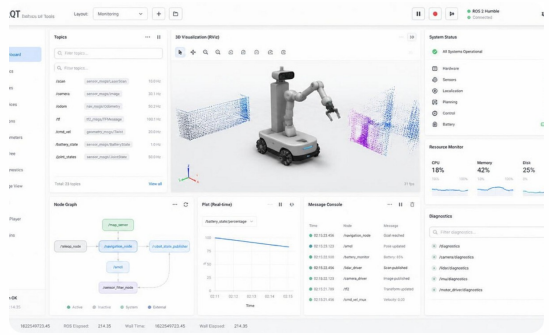
Python و C++ مدعومتان رسمياً

أدوات المحاكاة — Simulation Tools



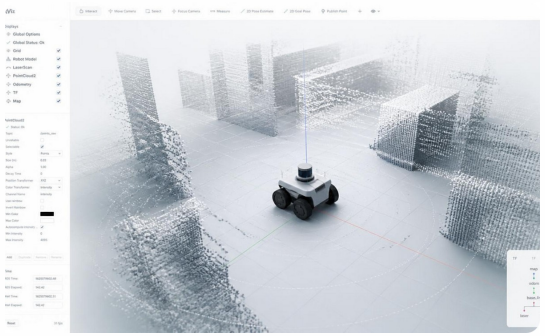
Stage

محاكاة ثنائية الأبعاد خفيفة وسريعة للاختبار



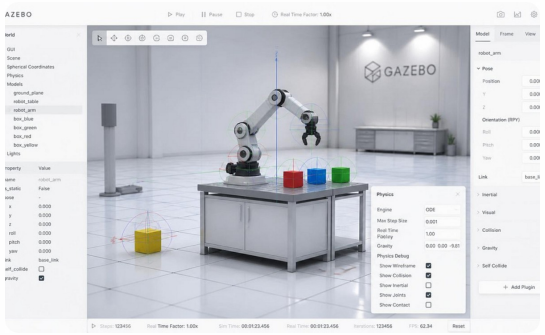
rqt

مجموعة أدوات رسومية متعددة لمراقبة وتشخيص النظام



RVIZ

تصور بيانات الحساسات والتخطيط في بيئة ثلاثية الأبعاد



Gazebo

محاكاة فيزيائية ثلاثية الأبعاد كاملة للروبوتات والبيئات

أوامر ROS الأساسية

`roscore`

تشغيل ROS Master — الخطوة الأولى دائماً

`rosrun <pkg> <node>`

تشغيل عقدة واحدة من حزمة معينة

`roslaunch <pkg> <file>`

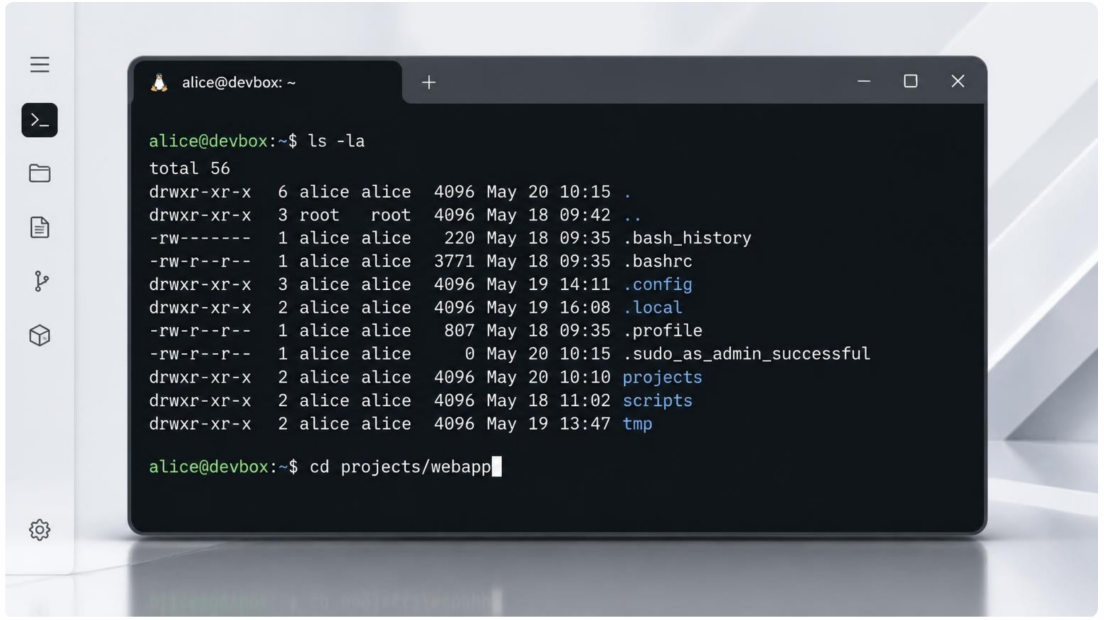
تشغيل ملف launch لعدة عقد دفعة واحدة

`rostopic list`

عرض قائمة جميع المواضيع النشطة

`rosnode list`

عرض قائمة جميع العقد الجارية حالياً



ملخص الدرس — ما تعلمناه اليوم

01

ثلاثة مستويات لنظام ROS

Filesystem ،Computation Graph ،Application

02

نظام الملفات ينظم الحزم

Catkin Workspace وهيكل package.xml
CMakeListsg

03

شبكة المعالجة تربط العقد

العقد تتواصل عبر ROS Master والمواضيع

04

للطلبServices للنشر، Topics

كل منهما له حالات استخدام مختلفة ومتكاملة

05

أدوات مساعدة متعددة

Gazebo، RVIZ، rqt، أوامر الطرفية

أسئلة للمراجعة — اختبر نفسك

1

السؤال الأول

ما هي المستويات الثلاثة لنظام ROS؟

2

السؤال الثاني

ما هو Catkin Workspace ولماذا نستخدمه؟

3

السؤال الثالث

ما الفرق الجوهري بين Topic و Service؟

4

السؤال الرابع

ما وظيفة ROS Master وماذا يحدث بدونه؟

5

السؤال الخامس

ما هي أداة rqt_graph وما فائدتها؟

الأسبوع القادم — ماذا سنتعلم؟

تثبيت نظام ROS

خطوات التثبيت الكاملة على Ubuntu

إعداد Ubuntu

تجهيز البيئة وضبط الإعدادات الأساسية

إعداد source.list

إضافة مستودعات ROS للنظام

GPG Keys

مفاتيح التحقق الأمنية للنظام

تنصيب ROS Noetic

التثبيت الكامل والتحقق من نجاحه

📌 واجب قبل الأسبوع القادم: ثبّت Ubuntu على جهازك الحقيقي أو على جهاز افتراضي (VM) ⚠